

SameGame in Java

An Object-Oriented Implementation Using the MVC and Observer
Patterns

Object-Oriented Programming — Project Report

Hayat Aodi, Erwann Lanteri-Enée, Rawan Meselmini, Salam Mohammad

Group 8

May 2026
Halmstad University

Contents

1	Introduction	2
2	Design	2
2.1	Architectural overview	2
2.2	Observer pattern	2
2.3	Model layer	3
2.4	View layer	3
2.5	Controller layer	4
3	Testing	4
3.1	Automated unit tests	4
3.2	Manual integration testing	5
4	Implementation Highlights	5
4.1	Flood-fill group detection	5
4.2	Decoupling view and controller	5
4.3	Hover preview without extra state	6
5	Results and Evaluation	6
5.1	Functionality	6
5.2	Extensibility	6
5.3	Limitations	7
6	Version Control	7
7	References	7

1 Introduction

SameGame is a classic single-player puzzle game originally released in 1985. The player is presented with a rectangular grid of coloured tiles. A *group* is a set of two or more tiles that share the same colour and are connected through horizontal or vertical adjacency (four-directional, never diagonal). Clicking a valid group removes all tiles in it, awards points, and causes the remaining tiles to fall downward under gravity; wholly empty columns then collapse to the left. The game ends when no valid group remains: if the board is completely empty the player wins, otherwise the player loses.

The project required producing a fully functional Java implementation of this game together with a clean, extensible design. The specific decisions taken here were:

- Use the **Model-View-Controller (MVC)** architectural pattern to separate game logic from rendering and input.
- Use the **Observer** pattern to decouple the model from its views, allowing multiple simultaneous renderers.
- Provide a **Swing GUI** as the primary view and a lightweight **console (ASCII) view** for debugging alongside it.
- Score groups with the quadratic formula $S = (n - 2)^2$, rewarding larger groups disproportionately.

Similar games such as *Bejeweled* or *Candy Crush* involve swapping adjacent tiles; SameGame is simpler in that only grouped removal occurs, making it a good fit for a semester-length project without sacrificing interesting design problems. The implementation covers all standard rules and adds a hover-highlight feature that previews which tiles will be removed before the player commits to a click.

2 Design

2.1 Architectural overview

The program is organised into five Java packages whose dependency graph is strictly layered: `observer` $\leftarrow$ `model` $\leftarrow$ `view` / `controller` $\leftarrow$ `Main`. No lower-level package imports a higher-level one; in particular, `model` contains zero references to `javax.swing` or any view class.

2.2 Observer pattern

Two interfaces in the `observer` package define the pattern contract:

GameSubject declares `addObserver`, `removeObserver`, and `notifyObservers`. **GameModel** implements it.

GameObserver declares a single method `update(GameModel)`. Both **SwingView** and **ConsoleView** implement it (via **GameView**, which extends **GameObserver** with lifecycle methods `display()` and `close()`).

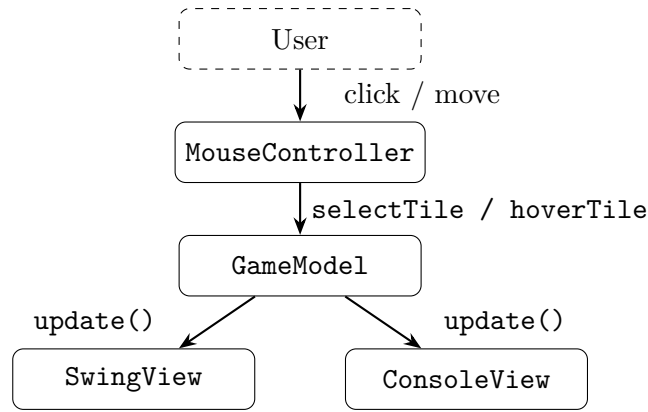


Figure 1: Data-flow diagram for a single user interaction.

The separation of `GameSubject` from `GameModel` means that any code that only needs to register observers can depend on the interface rather than the concrete class, limiting coupling.

2.3 Model layer

`GameModel` owns three pieces of mutable state: the `Board`, an integer score, and a `GameState` enum (`PLAYING`, `WIN`, `LOSE`). It also stores the *highlighted group*, a list of `int[] {row,col}` pairs that represent the tile group under the cursor; this is updated by `hoverTile()` and consumed by the view to paint a preview.

`Board` holds the two-dimensional `Tile[][]` grid and implements three algorithms:

1. **Flood-fill** (`findGroup`): a recursive depth-first search that collects all four-directionally connected tiles of the same colour starting from a seed position.
2. **Gravity** (`applyGravity`): after removal, each column is compacted bottom-up so non-empty tiles slide down to fill gaps.
3. **Column collapse** (`collapseColumns`): columns whose bottom cell is empty are removed and remaining columns shift left.

`Tile` is a simple mutable value object holding a colour integer (0 = empty, 1– N = game colour). Mutability is intentional: `Board` sets colour to 0 in-place during gravity and collapse rather than reallocating the entire grid.

2.4 View layer

Both views are *passive*: they read state from the model reference passed in `update()` but never mutate it. `SwingView` uses a custom inner class `BoardPanel` (`JPanel`) for painting. To avoid a circular import between the `view` and `controller` packages, `SwingView` exposes `getBoardComponent()`, which returns the board panel as a `JComponent`. The controller attaches its listeners to this component without needing to import the concrete `SwingView` type in the listener code.

2.5 Controller layer

`GameController` is an interface with two lifecycle methods, `attach()` and `detach()`. `MouseController` extends `MouseAdapter` (overriding only `mouseClicked` and `mouseMoved`) and implements `GameController`. Pixel-to-grid conversion uses the constant `SwingView.TILE_SIZE`:

$$\text{col} = \lfloor x_{\text{px}} / \text{TILE_SIZE} \rfloor, \quad \text{row} = \lfloor y_{\text{px}} / \text{TILE_SIZE} \rfloor$$

A new input method (keyboard, network, etc.) requires only implementing `GameController` and registering it in `Main`; no other class changes.

3 Testing

Testing was split into two complementary strategies.

3.1 Automated unit tests

JUnit 4 tests in `GameModelTest` cover the most critical model invariants, since the model is the layer with no external dependencies.

```
1 @Test
2 public void testInitialScoreIsZero() {
3     GameModel model = new GameModel(3);
4     assertEquals(0, model.getScore());
5 }
6
7 @Test
8 public void testGameStartsPlaying() {
9     GameModel model = new GameModel(3);
10    assertEquals(GameState.PLAYING, model.getState());
11 }
12
13 @Test
14 public void testRestartResetsScore() {
15     GameModel model = new GameModel(3);
16     model.restart(3);
17     assertEquals(0, model.getScore());
18 }
```

Listing 1: Excerpt from `GameModelTest.java`

The tests verify:

- Initial score is zero after construction.
- Initial game state is `PLAYING`.
- Calling `restart()` resets the score to zero.

3.2 Manual integration testing

Because `SwingView` depends on the Swing event dispatch thread, the GUI layer was tested manually:

1. Verified that clicking a single isolated tile (group size 1) has no effect on score or board state.
2. Verified that clicking a group of n tiles increases the score by exactly $(n - 2)^2$.
3. Verified gravity: after removal, tiles above the gap fall to the bottom of their column with correct relative order.
4. Verified column collapse: a column cleared entirely disappears and the columns to its right shift left.
5. Verified end conditions: the win overlay appeared when the last group was removed; the lose overlay appeared when no valid moves remained.
6. Verified the hover preview by moving the mouse over groups of varying sizes.

4 Implementation Highlights

4.1 Flood-fill group detection

The flood-fill in `Board.findGroup` is a recursive depth-first search. A key design choice was to return a *list* of positions rather than mutating the board in-place: the caller (`GameModel`) uses the same list for both scoring and the hover-preview highlight before any tile is actually removed.

```
1 private void floodFill(int r, int c, int color,
2                       boolean[][] visited, List<int[]> group) {
3     if (r < 0 || r >= rows || c < 0 || c >= cols) return;
4     if (visited[r][c] || grid[r][c].getColor() != color) return;
5     visited[r][c] = true;
6     group.add(new int[]{r, c});
7     floodFill(r - 1, c, color, visited, group);
8     floodFill(r + 1, c, color, visited, group);
9     floodFill(r, c - 1, color, visited, group);
10    floodFill(r, c + 1, color, visited, group);
11 }
```

Listing 2: Recursive flood-fill in `Board.java`

The `visited` array prevents the recursion from re-entering cells and bounds-checking is done at the top of each call, keeping the main logic clean.

4.2 Decoupling view and controller

A subtle problem arises with MVC in Swing: the controller needs to attach a `MouseListener` to the board panel, but if `MouseController` imports `SwingView` it creates a dependency from controller to view. The chosen solution was to expose the panel through the `JComponent` supertype:

```

1 // In SwingView (view package)
2 public JComponent getBoardComponent() { return boardPanel; }
3
4 // In MouseController (controller package)
5 public void attach() {
6     JComponent board = view.getBoardComponent();
7     board.addMouseListener(this);
8     board.addMouseMotionListener(this);
9 }

```

Listing 3: `SwingView.getBoardComponent()` breaks the cycle

Both packages depend only on `javax.swing.JComponent`; neither creates a dependency on the other’s concrete type at the listener-registration site.

4.3 Hover preview without extra state

The hover preview reuses the same flood-fill path as a real click. `GameModel.hoverTile()` stores the resulting group in `highlightedGroup`; `SwingView.BoardPanel.paintComponent()` reads it during every repaint and brightens those tiles. No separate “preview” state was introduced; the model’s highlighted-group field doubles as the preview channel at zero extra cost.

5 Results and Evaluation

5.1 Functionality

All specified requirements were implemented:

- Four-directional group detection with flood-fill.
- Quadratic scoring: $S = (n - 2)^2$.
- Gravity and column collapse after each removal.
- Win / lose detection after every move.
- Hover-preview highlight before committing a click.
- Restart button that resets score and generates a new board.
- Parallel `ConsoleView` for debugging.

5.2 Extensibility

The MVC + Observer design makes the most common extensions straightforward:

New view Implement `GameView` and call `model.addObserver(newView)` in `Main`. No changes to any existing class.

New controller Implement `GameController`, call `attach()` in `Main`. A keyboard controller, for example, would require zero changes to the model or views.

Board configuration Change `GameModel.ROWS`, `COLS`, or pass a different `numColors` value; no other changes needed.

5.3 Limitations

- No persistent high-score storage; the score resets on every run.
- Board size is fixed at compile time via constants rather than being configurable at runtime through a settings dialog.
- Deep flood-fills on large boards could overflow the call stack; an iterative implementation using an explicit `Deque` would be more robust.

6 Version Control

All development was tracked with Git. The working tree followed a single linear branch (`main`) with commits at meaningful milestones:

Hash	Message
99e1277	Init project
dd98aaa	SameGame V0
ff29a8e	Add HOW_TO_RUN.md for project setup and execution
dca12df	Updated SameGame project and added design improvements
8df6f57	Updated Board.java
0155159	Updated test
10e3440	Updated

The initial commit set up the package structure and build system. `dd98aaa` introduced a working but unpolished first version of all game rules. Subsequent commits refined the design patterns (observer separation, controller interface), added the hover-preview feature, and cleaned up Javadoc. Keeping commits small and tied to specific concerns made it easy to review which change introduced a particular behaviour when debugging.

7 References

References

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] J. Bloch, *Effective Java*, 3rd ed. Addison-Wesley, 2018.
- [3] Wikipedia contributors, “SameGame,” *Wikipedia, The Free Encyclopedia*, <https://en.wikipedia.org/wiki/SameGame>, accessed May 2026.
- [4] Oracle, “Creating a GUI With Swing,” *Java SE Documentation*, <https://docs.oracle.com/javase/tutorial/uiswing/>, accessed May 2026.
- [5] Oracle, “Java SE 17 API Specification,” <https://docs.oracle.com/en/java/docs/>, accessed May 2026.